Sim-to-Real Transfer for DroneRanger: Literature Review V2

Jiarui Zha, zha.j@northeastern.edu
CS 7980 Research Capstone, Northeastern University · DroneRanger hardware group
PI: Prof. Ilmi Yoon, NEURAI Lab · October 11, 2026

1. Purpose

DroneRanger is a low-cost autonomous drone being developed to patrol a household garden and, in the long run, to deter wildlife without contact. It is built on a Holybro X500 V2 airframe with PX4 on a Pixhawk 6C, an NVIDIA Jetson Orin Nano and an OAK-D Lite stereo camera. Its first paper claims that a single PX4–ROS 2 interface, shared by Gazebo, Unity and the real aircraft, "enables reliable simulation-to-reality transfer" [1]. The Spring 2026 prototype exposed what that interface had not been tested on [2]:

- about 0.5 m of altitude error near the ground;
- setpoints reported as reached while the drone was visibly offset;
- depth images with large invalid regions.

This semester the hardware group will rehearse every mission in PX4 software-in-the-loop (SITL) simulation before flying it. The flight milestones are manual outdoor flight (O1) and then an autonomous Mission-mode patrol over the garden (O2). Meanwhile, the reinforcement-learning (RL) group trains a navigation policy in Isaac Lab. This brief asks one practical question of the literature: *where will DroneRanger's simulators mislead it, and how can that gap be measured and narrowed?*

2. Method

I started from the project's own reference library, then searched IEEE Xplore, the Science and Nature journals, arXiv and Google Scholar in October 2026. I kept peer-reviewed work that either addresses sim-to-real transfer directly or quantifies a gap that a small PX4 multirotor faces: ground effect, wind, sensing, model parameters, software testing and evaluation. Aerial robots and work from 2017–2026 were preferred. Journal articles were resolved through the Northeastern University Library's LibKey service, and conference DOIs and the book were checked on Crossref.

Citation management. All entries are kept in a BibTeX file (references_brief.bib), which imports directly into Zotero, and are cited in IEEE style.

3. Annotated Sources

3.1 Journal articles

[3] The reality gap in robotics (Annu. Rev. Control Robot. Auton. Syst., 2026). This 2026 review is the most recent synthesis of the field. It traces the gap to four sources: dynamics; perception and sensing; actuation and control; and system design, which includes communication delay, control-frequency mismatch and privileged training signals. It also separates methods that *reduce* the gap (system identification, learned residuals, real-to-sim reconstruction) from methods that *overcome* it (randomization, adaptation, teacher–student training, modularity). **For DroneRanger:** all four sources are present. The X500 simulation model has no ground effect, and its depth camera has no noise. The RL environment replaces PX4 with a custom controller. Its policy decides at 50 Hz using ground truth that the real drone will not have, and it must eventually run over a serial link with real delays. The review is therefore a checklist for both the SITL twin and the RL hand-over.

[4] Sim-to-real discrepancies in PX4 flight logs (Sensors, 2026). The authors compared real PX4 flight logs with an untuned PX4 SITL simulation in Gazebo, both using the EKF2 estimator. Vertical-acceleration variance was about ten times larger in real flight (4.985 vs. 0.458), maximum tilt reached 72.1° instead of 24.0°, and GPS and magnetometer fluctuations appeared only in reality. The airframes and PX4 versions differed, which the authors flag as confounds. **For DroneRanger:** this is the closest published analogue to the hardware group's SITL twin, with the same autopilot,

simulator and estimator, and it shows that a default SITL model is optimistic. The twin must be tuned with the real X500's parameters and noise before its predictions are trusted. Comparisons should use matched firmware versions.

[5] Sim2Real predictivity (IEEE Robot. Autom. Lett., 2020). Nine navigation models were run both in a 3-D-scanned replica of a lab and on a real robot. The authors introduced the Sim-vs-Real Correlation Coefficient (SRCC) to measure how well simulated results predict real ones. The simulator used in a public challenge had an SRCC of only 0.18 for success, because agents exploited imperfect collision handling to slide along walls. Tuning the simulator raised the SRCC to 0.844. **For DroneRanger:** the CCWC paper asserts reliable transfer without any such measurement. Flying the same mission and parameter files in SITL and in the field, then comparing hover error, waypoint error and failsafe timing, turns that assertion into a number.

[6] Robot learning from randomized simulations (Front. Robot. AI, 2022). This review organizes domain randomization into static, adaptive and adversarial variants. It concludes that policies trained with randomization are consistently more robust than policies trained on a single nominal model. It also warns that initial guesses of the simulator parameters and their variances are typically very inaccurate, and that the methods rarely work out of the box. **For DroneRanger:** the review gives a principled way to choose what to vary in Monte Carlo SITL runs: mass (the computer and camera add about 0.37 kg), wind, GNSS bias and link delay. Its warning is a reason to ground those ranges in the hardware group's own flight logs first.

[7] Champion-level drone racing (Nature, 2023). Swift, a quadrotor policy trained with deep RL, won 15 of 25 races against three world-champion pilots and set the fastest race time. Its simulator used an aerodynamic model fitted to flight data instead of randomizing the dynamics. After a few real runs recorded with motion capture, the team fitted residual models of two errors: drift in the visual–inertial odometry, using Gaussian processes, and unmodelled dynamics, using nearest-neighbour regression. They then fine-tuned the policy in the augmented simulator. **For DroneRanger:** a small amount of real data with independent ground truth made Swift's simulator trustworthy. The equivalent here is to log every flight against rangefinder and ground-marker truth, and to feed the measured errors back into the SITL model.

[8] Learning high-speed flight in the wild (Sci. Robot., 2021). A quadrotor flew through forests, snow-covered terrain and ruins using a policy trained only in simulation. It transferred zero-shot: there were no crashes at 3 and 5 m/s, 8 of 10 runs succeeded at 7 m/s, and 60% succeeded at 10 m/s. Two choices made this possible. First, a privileged expert supervised a student that saw only onboard sensors. Second, the student's training depth was computed by stereo matching on simulated image pairs, so it had the same kind of noise as the real stereo camera. **For DroneRanger:** its Gazebo and Isaac simulators render perfect depth. The OAK-D Lite, by contrast, returns invalid pixels below about 0.35 m and false far readings, so the depth interface given to the algorithm and RL groups should be simulated with these failure modes.

[9] Neural-Fly (Sci. Robot., 2022). Neural-Fly learned a wind-invariant representation of aerodynamic forces from 12 minutes of flight data and adapted only its output coefficients online. It outperformed nonlinear, INDI and L1-adaptive baselines in winds of up to 12.1 m/s. Its outdoor vehicle is close to DroneRanger in design: a Pixhawk running PX4 in Offboard mode under a companion computer, with GPS for position. **For DroneRanger:** wind is the largest dynamics gap that its default SITL worlds leave out. At San Jose airport, about one autumn day in five has a gust of 25 mph or more. Missions should therefore be rehearsed in PX4's windy world, flights should have a gust go/no-go limit, and wind data should be logged for a later adaptive layer.

[10] SOUS VIDE (IEEE Robot. Autom. Lett., 2025). SOUS VIDE built a drone simulator on a Gaussian-splatting reconstruction of a real scene, made from a 2–3-minute hand-held video. A privileged, randomized expert controller was distilled into an onboard visuomotor policy with a rapid-adaptation module. In 105 hardware experiments the policy flew zero-shot and was robust to a 30% mass increase and to strong gusts; it failed only when the light fell below about 40% of the original level. It ran on the same companion computer as DroneRanger, a Jetson Orin Nano. **For DroneRanger:** it is the template for a future garden digital twin, which could replace the parking-lot scene now used in Isaac Sim. The twin is a static snapshot, though, and capture must stay inside the host property.

3.2 Conference papers

[1] DroneRanger (IEEE CCWC, 2026). The project's own paper presents the architecture under review. PX4 handles estimation and control, and ROS 2 handles mission logic. A Micro XRCE-DDS bridge exposes both through the same topics in Gazebo, in Unity and on the real aircraft. The architecture also includes a finite-state mission executive and depth-based avoidance. The paper validated behaviours qualitatively and listed latency audits, hardware-in-the-loop tests and multi-run statistics as future work. **For DroneRanger:** this review treats the paper's transfer claim as the hypothesis this semester must test.

[11] Neural Lander (IEEE ICRA, 2019). Neural Lander learned the ground-effect force on a 1.47 kg quadrotor from flight data, using a spectrally normalized neural network. It added this force to a nonlinear controller with stability guarantees. Landing height error fell from 0.13 m to almost zero, and lateral drift fell by up to 90%. **For DroneRanger:** the Spring 2026 prototype had about 0.5 m of altitude error near 1 m above the ground, and the PX4 Gazebo X500 model has no ground-effect term at all. Near-ground altitude is therefore where SITL and flight will disagree first. The TFmini Plus rangefinder should be logged unfused, as independent truth, before it is fused into EKF2.

[12] Data-driven system identification of quadrotors (IEEE/RSJ IROS, 2024). Eschmann et al. identified a quadrotor's inertia, thrust curve, torque coefficient and motor delay from about one minute of ordinary flight data, without a thrust stand. They released a tool that reads PX4 ULog files. An RL policy trained in simulation on parameters from only 73 s of data then flew a 3.35 kg quadrotor outdoors in gusts of up to 8 m/s. **For DroneRanger:** this is the cheapest route from the real X500 to a calibrated Gazebo model. The logs of the first indoor hover tests can supply the thrust and motor-delay values that the default model only guesses.

[13] Can simulation detect robotics bugs early? (IEEE ICST, 2018). Timperley et al. analysed 228 bug fixes in the ArduPilot autopilot. Only 10 of the bugs needed physical hardware to reproduce. Most could be triggered in software-in-the-loop simulation, provided the tests varied configurations and inputs, since configuration (81 bugs) and environment, such as wind (22 bugs), were the common triggers. **For DroneRanger:** this supports exercising every mission, parameter change and failsafe in PX4 SITL first (O0), with runs swept over parameter files and wind settings rather than run once. It also marks what SITL will miss: hardware faults and concurrency problems. Those need bench tests and staged field tests.

3.3 Book

[14] Introduction to Multicopter Design and Control (Springer, 2017). This textbook models multicopters from first principles. It also devotes chapters to measuring what a simulator needs: propulsion-system modelling, the dynamic model and parameter measurement, sensor calibration and measurement models, Kalman filtering and state estimation, and health evaluation and failsafe design. **For DroneRanger:** it is the manual for the "identify before you randomize" step: weighing the roughly 2.1 kg aircraft, fitting thrust and motor time constants, and characterizing IMU, barometer and GNSS noise, so that the Gazebo X500 model matches the real vehicle.

4. Synthesis for DroneRanger

The sources agree on three points. First, transfer improves when the simulator is calibrated with a small amount of real data [7], [11], [12], [14]. Second, the uncertainty that remains should be randomized [6]. Third, transfer has to be measured rather than assumed [3]–[5]. Table 1 maps these lessons onto this semester.

Table 1. Sim-to-real gaps for DroneRanger and the semester response.

Gap	Evidence	Response this semester	Measured by
Ground effect and altitude	[4], [11]	Log the TFmini Plus unfused as truth, then fuse it in EKF2; compare near-ground hover in SITL and in flight	Hover altitude RMSE ≤ 0.15 m
Model parameters	[12], [14]	Identify mass, thrust and motor delay from flight logs; update the Gazebo X500 model	Hover throttle and step response, sim vs. real
Wind	[9]	Rehearse in the windy world; set a gust go/no-go limit; log wind	Tracking error vs. gust
Depth and GNSS sensing	[7], [8]	Model invalid depth and GNSS bias from logs; judge waypoints against surveyed markers	Invalid-pixel fraction; waypoint error

Gap	Evidence	Response this semester	Measured by
Software, timing and control stack	[3], [13]	Run every mission and failsafe in SITL; bench-test the serial link; run the RL policy through PX4 SITL	Faults caught before flight; latency distribution
What to randomize	[6]	Monte Carlo SITL over wind, mass, GNSS bias and delay	Geofence margin; mission success rate
Evaluating the claim	[1], [5]	Pair every SITL run with a real run, with at least ten trials per metric	Sim–real difference per metric
Garden scene (future)	[10]	Capture a splat twin of the host garden only	—

For the hardware group, simulation stays the first gate: no mission flies until it has passed SITL. But simulation is not the final word. What this semester can claim is the measured difference between SITL and the garden, and narrowing that difference with real data is the platform's contribution to the RL policy and, later, to learned deterrence.

References

[1] Z. Ling *et al.*, "DroneRanger: Simulation-guided UAV framework for autonomous wildlife deterrence," in *Proc. IEEE 16th Annu. Comput. Commun. Workshop Conf. (CCWC)*, 2026, pp. 1221–1227, doi: 10.1109/CCWC67433.2026.11393823.

[2] R. Yin, "Drone project technical handover: PX4 / ROS 2 / OAK-D Lite autonomous flight and obstacle-aware planning system," NEURAI Lab, Northeastern Univ., San Jose, CA, USA, Internal tech. rep., 2026.

[3] E. Aljalbout *et al.*, "The reality gap in robotics: Challenges, solutions, and best practices," *Annu. Rev. Control Robot. Auton. Syst.*, vol. 9, no. 1, pp. 403–432, May 2026, doi: 10.1146/annurev-control-031924-100130.

[4] K. Kose, J. Wing, N. A. Kose, C. Guadarrama-Trejo, A. Sowers, and A. Rasheed, "A UAV testbed for diagnosing hardware vulnerabilities: Quantifying sim-to-real discrepancies in PX4 flight logs," *Sensors*, vol. 26, no. 10, Art. no. 3188, May 2026, doi: 10.3390/s26103188.

[5] A. Kadian *et al.*, "Sim2Real predictivity: Does evaluation in simulation predict real-world performance?" *IEEE Robot. Autom. Lett.*, vol. 5, no. 4, pp. 6670–6677, Oct. 2020, doi: 10.1109/LRA.2020.3013848.

[6] F. Muratore, F. Ramos, G. Turk, W. Yu, M. Gienger, and J. Peters, "Robot learning from randomized simulations: A review," *Front. Robot. AI*, vol. 9, Art. no. 799893, Apr. 2022, doi: 10.3389/frobt.2022.799893.

[7] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, and D. Scaramuzza, "Champion-level drone racing using deep reinforcement learning," *Nature*, vol. 620, no. 7976, pp. 982–987, Aug. 2023, doi: 10.1038/s41586-023-06419-4.

[8] A. Loquercio, E. Kaufmann, R. Ranftl, M. Müller, V. Koltun, and D. Scaramuzza, "Learning high-speed flight in the wild," *Sci. Robot.*, vol. 6, no. 59, Art. no. eabg5810, Oct. 2021, doi: 10.1126/scirobotics.abg5810.

[9] M. O'Connell *et al.*, "Neural-Fly enables rapid learning for agile flight in strong winds," *Sci. Robot.*, vol. 7, no. 66, Art. no. eabm6597, May 2022, doi: 10.1126/scirobotics.abm6597.

[10] J. Low, M. Adang, J. Yu, K. Nagami, and M. Schwager, "SOUS VIDE: Cooking visual drone navigation policies in a Gaussian splatting vacuum," *IEEE Robot. Autom. Lett.*, vol. 10, no. 5, pp. 5122–5129, May 2025, doi: 10.1109/LRA.2025.3553785.

[11] G. Shi *et al.*, "Neural Lander: Stable drone landing control using learned dynamics," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2019, pp. 9784–9790, doi: 10.1109/ICRA.2019.8794351.

[12] J. Eschmann, D. Albani, and G. Loianno, "Data-driven system identification of quadrotors subject to motor delays," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2024, pp. 8095–8102, doi: 10.1109/IROS58592.2024.10801441.

[13] C. S. Timperley, A. Afzal, D. S. Katz, J. M. Hernandez, and C. Le Goues, "Crashing simulated planes is cheap: Can simulation detect robotics bugs early?" in *Proc. IEEE Int. Conf. Softw. Testing, Verification Validation (ICST)*, 2018, pp. 331–342, doi: 10.1109/ICST.2018.00040.

[14] Q. Quan, *Introduction to Multicopter Design and Control*. Singapore: Springer, 2017, doi: 10.1007/978-981-10-3382-7.